

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 - Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 - Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	<i>Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.</i>		
Student Name:	JEEVA RAGHUL .M	Reg. No.:	192524487
Section / Batch:	B-TECH AIDS	Date:	17/07/2026

Code of Conduct

I _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: _____

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

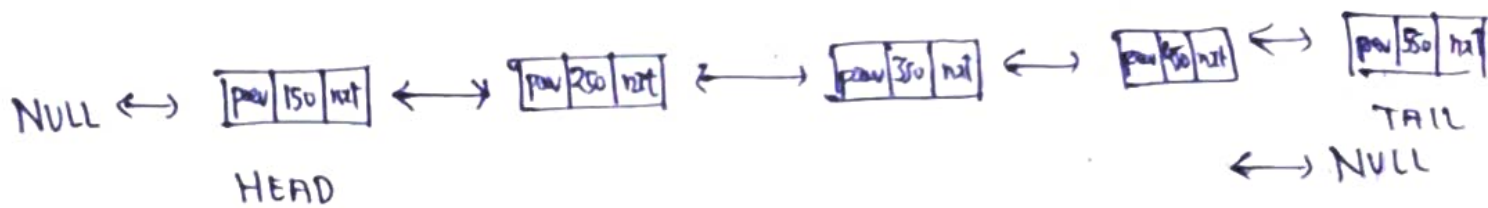
Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:		100 Marks	

① Insert the following elements 150, 250, 350, 450, 550 into the double linked list and delete the elements 150 and 250 from the double linked list. Show the linked structure using a node diagram before and after delete operations.

= Each node of a doubly linked list has three fields

- prev - pointer / address of the previous node.
- data - the value stored in the node.
- next - pointer / address of the next node.

The five elements are inserted at the end of the list, one after another, giving the following structure.



DELETE OPERATION :-

150 is the HEAD Node. To delete it

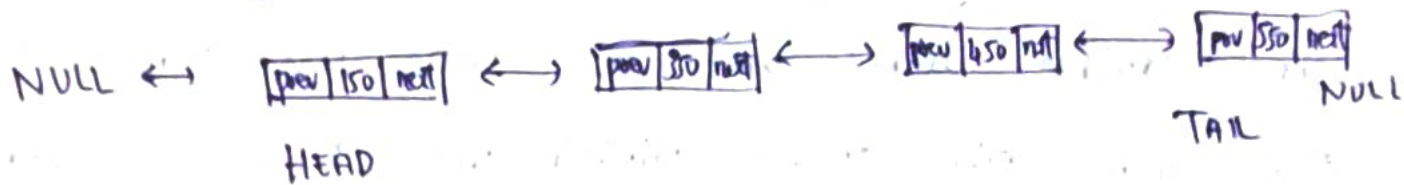
temp = HEAD

HEAD = HEAD -> next

HEAD -> prev = NULL

free(temp)

After deleting 150:



Delete Operation 2:

250 is the head node (Since 150 was already removed).

To delete it.

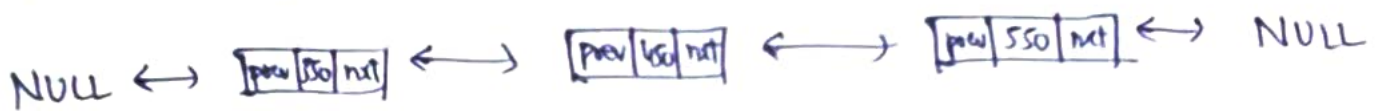
temp = HEAD

HEAD = HEAD → next

HEAD → prev = NULL

free (temp)

After Deleting:



General Deletion Algorithm:

delete (value):

If HEAD == NULL : return

temp = HEAD

while temp != NULL and temp.data != value:

temp = temp.next.

If temp == NULL: return

if temp.prev != NULL:

temp.prev.next = temp.next

else:

HEAD = temp.next.

If temp.next != NULL:

temp.next.prev = temp.prev

else:

TAIL = temp.prev

free(temp).

② Mr. XYZ booked a ticket on Pandian Express. He reached the station and was looking for his PNR number in the chart. Call out binary search procedure to search the PNR number 4056786. The ticket numbers are 4056779, 4056780, 4056781, 4056782, 4056783, 4056784, 4056785, 4056786, 4056787, 4056788, 4056789.

= The 11 PNR numbers, indexed from 0 to 10:

Field	0	1	2	3	4	5	6	7
PNR	4056779	4056780	4056781	4056782	4056783	4056784	4056785	

7	8	9	10
4056786	4056787	4056788	4056789

Binary search repeatedly compares the target with the middle element of the current search range and eliminates half the remaining entries at each step.

binary Search (arr, target):

low = 0

high = arr.length - 1

while low <= high:

mid = (low + high) / 2

if arr[mid] == target:

return mid

else if arr[mid] < target:

low = mid + 1

else:

high = mid - 1

return -1

Applying algorithm to search for 4056786:

Step 1 - mid = 5, arr[5] = 4056784 (< target). Search shifts right

Step 2 - new range [6, 10], mid = 8, arr[8] = 4056787 (> target).
Search shifts left.

Step 3 - new range [6, 7], mid = 6, arr[6] = 4056785 (< target).
Search shifts right.

Step 4 - new range [7, 7], mid = 7, arr[7] = 4056786 = target.
PNR found.

This is far more efficient than linear search, which could take up to 11 comparisons to find the same PNR.